

SKILL-01

To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile

```
int:
int:  git branch -m <name>
Initialized empty Git repository in /root/SkillWeek1/.git/
root@SRIVALLI:~/SkillWeek1# git status
On branch master

no commits yet

nothing to commit (create/copy files and use "git add" to track)
root@SRIVALLI:~/SkillWeek1# nano hello.c
root@SRIVALLI:~/SkillWeek1# gcc hello.c -o hello
root@SRIVALLI:~/SkillWeek1# ./hello
Hello Linux
root@SRIVALLI:~/SkillWeek1# nano Makefile
root@SRIVALLI:~/SkillWeek1# make
gcc hello.c -o hello
root@SRIVALLI:~/SkillWeek1# make run
./hello
Hello Linux
root@SRIVALLI:~/SkillWeek1# make clean
rm -f hello
root@SRIVALLI:~/SkillWeek1# pwd
~/SkillWeek1
makefile bin hello.c include src
root
SRIVALLI
Thu Aug 20 04:58:03 UTC 2026
root@SRIVALLI:~/SkillWeek1# |
```

Report: In this task, I installed and configured the Linux environment and checked GCC and Git installation. I created a project folder structure and initialized a Git repository. I also learned the basic working of the Linux shell and created a simple Makefile to compile and run a C program. This task helped me understand the basic Linux development environment and tools. Linux environment, GCC, Git, project structure, and Makefile were successfully created and tested.

To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing

```
top - 05:21:38 up 34 min, 1 user, load average: 0.03, 0.05, 0.01
Tasks: 24 total, 1 running, 23 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.
MiB Mem : 7784.6 total, 6957.4 free, 535.8 used, 450.2 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 7248.8 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+
1992	root	20	0	9300	5684	3496	R	0.3	0.1	0:00.01
1	root	20	0	21980	13508	9772	S	0.0	0.2	0:01.44
2	root	20	0	3180	2204	2072	S	0.0	0.0	0:00.00
6	root	20	0	3196	2128	2008	S	0.0	0.0	0:00.00
178	root	20	0	4244	2680	2432	S	0.0	0.0	0:00.01
179	message+	20	0	9892	5672	4704	S	0.0	0.1	0:00.27
191	root	20	0	17980	8864	7812	S	0.0	0.1	0:00.15
217	syslog	20	0	222516	6056	4676	S	0.0	0.1	0:00.08
220	root	20	0	3124	1972	1836	S	0.0	0.0	0:00.01
226	root	20	0	107036	23184	13732	S	0.0	0.3	0:00.15
320	root	20	0	3184	1108	980	S	0.0	0.0	0:00.00
321	root	20	0	3200	1244	1108	S	0.0	0.0	0:00.07
324	root	20	0	6076	5352	3652	S	0.0	0.1	0:00.05
328	root	20	0	6676	4500	3728	S	0.0	0.1	0:00.01
374	root	20	0	20052	11124	9312	S	0.0	0.1	0:00.13
375	root	20	0	21176	3500	1784	S	0.0	0.0	0:00.00
397	root	20	0	6080	5304	3672	S	0.0	0.1	0:00.00
673	polkitd	20	0	308172	8228	7312	S	0.0	0.1	0:00.08
1175	root	19	-1	34184	12632	11592	S	0.0	0.2	0:00.08
1227	systemd+	20	0	91036	8012	7044	S	0.0	0.1	0:00.06
1282	root	20	0	24888	6272	5168	S	0.0	0.1	0:00.26
1334	systemd+	20	0	21344	13216	11092	S	0.0	0.2	0:00.04
1833	root	20	0	1830360	13788	11448	S	0.0	0.2	0:00.04
1948	root	20	0	370104	21068	18080	S	0.0	0.3	0:00.02

Report:

In this task, I learned how Linux creates and manages processes using fork() and exec(). I observed the relationship between parent and child processes by displaying their PIDs and PPIDs. Commands like ps, pstree, and top were used to view running processes and process hierarchies. I also used strace to trace system calls and understand how a process interacts with the operating system. The process creation, parent-child relationship, process tree, and system call tracing were successfully observed and analyzed.

To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

```
root@SRIVALLI:~/SkillWeek1# cat my_shell.c
#include <stdio.h>

int main() {
    printf("Hello World\n");
    return 0;
}gcc my_shell.c -o my_shell
```

Report: In this task, I created a simple interactive shell using a loop. The program displays a prompt, accepts user input, and checks for the exit condition. Different commands were tested to verify the working of the interactive loop. This task helped me understand how a shell continuously accepts and processes user commands.

The interactive loop was successfully implemented and tested with different commands and exit conditions.

To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction

```

false                                ulimit [-SHabco
fc [-e ename] [-lnr] [first] [last]> umask [-p] [-S]
fg [job_spec]                       unalias [-a] na
for NAME [in WORDS ... ] ; do COMM> unset [-f] [-v]
for (( exp1; exp2; exp3 )); do COMM> until COMMANDS;
function name { COMMANDS ; } or nam> variables - Nam
getopts optstring name [arg ...]    wait [-fn] [-p
hash [-lr] [-p pathname] [-dt] [nam> while COMMANDS;
help [-dms] [pattern ...]          { COMMANDS ; }

root@SRIVALLI:~/SkillWeek1# helloo
Command 'helloo' not found, did you mean:
  command 'hello' from snap hello (2.10)
  command 'hellow' from snap hellow (0.1)
  command 'hello' from deb hello (2.10-3)
  command 'hello' from deb hello-traditional (2.10-6)
See 'snap info <snapname>' for additional versions.
root@SRIVALLI:~/SkillWeek1# cat input_handler.c
gcc input_handler.c -o input_handler
./input_handler

root@SRIVALLI:~/SkillWeek1# ./input_handler
-bash: ./input_handler: No such file or directory
root@SRIVALLI:~/SkillWeek1# |

```

Report:

In this task, I developed a program to capture keyboard input and process user commands. The program handled Enter key input, managed the input buffer, and supported multi-character commands. User interaction was tested by entering different commands and verifying the program response.

Keyboard input was captured successfully, and the program correctly processed user commands and user interactions.